技术尽调报告 • 2026-10-25

开源图像处理项目

对比分析报告

本地 fdxmp × LightCraft × darktable

架构、色彩科学、工程质量与许可边界

> **范围**：先剖析 D:\color-adjustment-software 下的本地项目 fdxmp（v1.6.1），再分别分析 storytold/lightcraft 与 darktable-org/darktable，最后给出相同点、各自优缺点与对 fdxmp 的建议。**本报告不修改任何代码。**

分析对象	fdxmp v1.6.1（本地） / LightCraft v0.13.0 / darktable master（5.8.0-dev，稳定版 5.6.2）
数据截止	2026-10-25（Asia/Shanghai）
方法	仓库静态审查 + 代码度量 + 官方文档 + GitHub API；未重新编译 fdxmp，未做跨软件画质实拍对比
许可	fdxmp：MIT；LightCraft：MIT OR Apache-2.0；darktable：GPLv3

本报告不修改任何代码。所有结论基于 2026-10-25 的仓库快照；性能数字凡来自项目文档的均标注 [自述]。

执行摘要

把三个项目放在一起看，第一件重要的事不是“谁功能多”，而是**它们不在同一条时间线上。** darktable 已经迭代 14 年，LightCraft 用 25 天做出了一个 0.13.0 的全流程 alpha，fdxmp 则只有 15 天、89 次提交。任何忽略“年龄”的功能对比都会失真；同样，任何忽略“许可”的代码借鉴都会带来法律风险。

五个结论

01 代际差是第一位的事实。 darktable 的 531.9k 行 C/H、97 个模块、14 年相机/镜头数据，与 LightCraft 的 316.1k 行 Rust、4,912 个测试、25 天冲到 14.6k stars，代表两种完全不同的工程范式；fdxmp 的 34.3k 行 Rust 加 21.3k 行移动端代码，则是“移动优先 + AI 修图”的第三种路线。

02 相同点比想象中多。 三者都坚持非破坏编辑、RAW 解码、场景参考线性工作空间、GPU/CPU 双路径、局部蒙版、预设/历史、跨平台桌面与本地优先。差别主要在“深度、覆盖、成熟度”，而不是“有没有这个功能”。

03 darktable 的护城河是深度与生态，不是 UI。 97 个 iop 模块、46 个 OpenCL 内核、colorbalancergb / toneequal / sigmoid / AgX / spektrafilm 等色彩工具、Lua 扩展与 14 年 RAW 样本积累，构成很高的复制门槛；代价是 GPLv3、GTK3 的陡峭交互和较重的资源占用。

04 LightCraft 最值得学的是工程方法。 强制分层、生产代码禁 unwrap/unsafe、CPU/GPU 1/255 一致性、4,912 个测试、每条命令可被 CLI/MCP 驱动，再加上 MIT/Apache 许可，使它既是一个可研究的产品，也是一个可合法复用的代码库。但它仍是 alpha：官方自评“功能广度 81.6%、真正可投入工作约 61%”，相机色彩与 RAW 覆盖是硬缺口。

05 fdxmp 的胜负手不在“再加一个滑杆”。 四端覆盖、AI 修图链、MIT + C ABI 是它真正的差异化；但相机色彩（无 DCP / as-shot WB）、图库/持久化、可复现的 CI 与文档一致性是三大地基。先补地基，再谈“对标 Lightroom”，否则功能表越漂亮，越容易被一次实拍对比打回原形。

关键指标一览

指标	fdxmp	LightCraft	darktable
版本 / 年龄	v1.6.1 / 15 天	v0.13.0 / 25 天	5.6.2 稳定 / 5.8.0-dev / 14 年
代码规模	Rust 34.3k + 移动端 21.3k 行	Rust 316.1k 行	C/C++ 531.9k 行（不含子模块）
许可	MIT	MIT OR Apache-2.0	GPLv3
测试	305 个（11 个 ignored）	4,912 个	CTest + 独立集成测试子模块
社区指标	Gitee 远端；未见公开 stars	14.6k stars / 1,059 open issues	13.3k stars / 651 open issues
平台	Windows + iOS / Android / HarmonyOS	macOS / Windows / Linux / Web / FreeBSD	Linux / Windows / macOS / FreeBSD
工作空间	线性 sRGB（f16）	线性 Rec.2020	线性 Rec.2020（scene-referred）
一句话定位	移动端 + AI 修图	桌面全流程 + 自动化	模块深度 + 生态

一句话选型

如果你的目标是……	优先选择	原因
成熟、极限画质、最大 RAW/镜头覆盖	darktable	14 年积累、97 个模块、RawSpeed/LibRaw、Lua 生态；接受 GPLv3 与学习曲线。
现代桌面全流程、可复用代码、自动化	LightCraft	MIT/Apache、分层 Rust、4,912 测试、MCP/CLI；接受 alpha 稳定性。
移动端、端侧 AI 修图、MIT 嵌入	fdxmp	四端覆盖、AI 修图链、C ABI；先补相机色彩、图库与验证。

> **阅读提示**：报告中的性能数字，凡来自项目 README/阶段报告的，均标注“[自述]”；本报告没有重新编译或跑分，因此不把它们当作独立实测。代码行数为物理行（含空行、注释与测试）。

1 范围、方法与口径

1.1 三个分析对象

fdxmp（本地项目）：位于 `D:\color-adjustment-software`，Cargo workspace 共 8 个 crate，版本 1.6.1；Gitee 远端 `fish-ling/color-adjustment-software`；git 历史从 2026-10-09 到 2026-10-24，共 89 次提交。README 自述目标是“跨平台专业图像处理软件（RAW 解码 / 色彩科学 / GPU 渲染），移动端完全对标 Lightroom”。

LightCraft：GitHub `storytold/lightcraft`，v0.13.0 于 2026-10-23 发布，仓库创建于 2026-09-30；截至 2026-10-25 有 14,551 stars、5,595 forks、1,059 open issues。纯 Rust，29 个 crate + 3 个 app + xtask；核心开发分支 `main`，本次审查的浅克隆 HEAD 为 `38f8d25`（2026-10-24）。

darktable：GitHub `darktable-org/darktable`，2012-03-21 创建；最新稳定版 5.6.2（2026-10-04），master 的 `RELEASE_NOTES.md` 已指向 5.8.0 开发版。13,322 stars、1,495 forks、651 open issues；C/GTK3 技术栈，`src/` 下 571 个 `.c` 与 341 个 `.h`，97 个 iop 模块。

1.2 证据来源与分析方法

维度	做法
代码度量	浅克隆/稀疏检出，统计文件、行数、测试、模块；fdxmp 另计 Swift/Kotlin/ArkTS。
代码走查	fdxmp `core/color/ffi`；LightCraft `pipeline/gpu/catalog`；darktable `develop/iop/iop_order`。
文档与版本	README、架构、路线图、缺口清单、阶段报告、修复日志、发布说明。
公开数据	GitHub API：stars/forks/issues/license/创建时间/发布。
交叉验证	用代码核对文档中的版本、测试、产物、FFI 数量等关键声明。

1.3 口径与限制

- **没有重新编译 fdxmp**：仓库内没有 `target/`，本机 PATH 也没有 cargo；报告中的测试/性能是静态统计或文档自述，不等同于复现。
- **代码行数不可跨语言直接等价**：Rust/C 的抽象密度不同；这里只用它说明“工程量级”，不做生产率比较。
- **darktable 不含子模块**：RawSpeed、LibRaw、OpenCL-Headers、Lua scripts 等通过 git submodule 引入，统计的 531.9k 行只是主仓 `src/`。
- **LightCraft 的成熟度数字是自评**：81.6% 功能广度、约 61% “ready for real work” 来自其 `ROADMAP.md`；本报告将其视为方向性指标。
- **stars/issues 是时点快照**：2026-10-25 的 GitHub 数据，会随社区活动变化；不能单独代表代码质量。
- **许可是工程判断，不是法律意见**：涉及 LGPL/GPL 的具体分发方式，应由法务复核。

1.4 术语表

术语	含义
非破坏编辑	原始像素不被改写；编辑以参数/历史/侧车文件保存，可随时回退。
场景参考（scene-referred）	在显示编码前，以线性光/场景亮度为工作空间；高光余量更大，色调映射集中在后段。
工作空间	管线内部统一使用的 RGB 色域与传递函数；fdxmp 为线性 sRGB，LightCraft/darktable 为线性 Rec.2020。
DCP	DNG Camera Profile，描述相机传感器色彩到标准色域的映射；Lightroom 的“相机标准/中性”依赖它。
As-shot WB	RAW 文件记录的拍摄白平衡；缺失时只能以固定参考温度近似，导致开尔文刻度与厂商不一致。
DAM / Catalog	数字资产管理/图库数据库；保存导入记录、评级、相册、编辑历史，而不仅是单个文件的参数。
OpenCL / wgpu	两种 GPU 计算接口；darktable 用 OpenCL，fdxmp/LightCraft 用 wgpu（Metal/Vulkan/DX12）。

2 本地项目 fdxmp：移动优先的“第四种路线”

2.1 项目快照

项目项	现状
仓库 / 版本	`D:\color-adjustment-software`；Cargo workspace v1.6.1；Gitee 远端
历史	2026-10-09 首提交，2026-10-24 最后提交；89 commits
代码	Rust 34.3k 行（core 23.7k、win_ui 6.1k、ffi 2.3k、color 1.6k）；Swift 14.3k、Kotlin 6.3k、ArkTS 0.7k
结构	8 个 workspace 成员：core / color / ffi / cli / win_ui / android_bridge / ohos_bridge / packer_ui
测试	305 个 `#[test]`（含 62 个集成测试、11 个 ignored；`crates/color` 37 个）
构建产物	仓库内 `dist/` 仅有 `.gitkeep`；未见安装包/APK/HAP 实体（文档曾描述已生成）
CI	仅 `.github/workflows/ios-ipa.yml` 一个 iOS 云打包工作流；未见 Windows/Android 持续集成

2.2 架构：一个核心，四端外壳

fdxmp 的分层非常清楚：**平台 UI → 桥接层 → Rust 核心 → GPU/CPU 渲染。**Windows 用 egui/eframe，iOS 用 SwiftUI，Android 用 Jetpack Compose，HarmonyOS 用 ArkTS；四端都不复制业务逻辑，而是通过 C ABI、JNI 或 N-API 调同一套 `fdxmp-core`。

层	组成	职责
平台层	Windows egui / iOS SwiftUI / Android Compose / Harmony ArkTS	交互、相册、分享、平台权限
桥接层	`fdxmp-ffi`（C ABI）、`android_bridge`（JNI）、`ohos_bridge`（N-API）	字符串/字节缓冲区所有权、错误模型、panic 捕获

层	组成	职责
核心层	`decode`、`params`、`pipeline`、`color_engine`、`tone`、`masks`、`smartmask`、`retouch`、`preset`、`export`、`history`、`cache`	解码、调色、蒙版、AI、预设、导出、历史
渲染层	`gpu_pipeline` + WGSL（geometry / base / blur / main / encode / overlay）	wgpu compute；CPU 参考路径用于移动端预览与导出

渲染管线可以概括为：**几何 → 基底（白平衡 → 相机校准 → 曝光）→ 两趟模糊 → 主调色（色调 → 曲线 → HSL/分级 → 细节）→ 逐蒙版合成 → 显示编码**。中间纹理是 `rgba16float` 线性 sRGB，只有最后一次编码量化到 8bit；这一点与 LightCraft/darktable 的 scene-referred 思路一致，但工作色域比它们窄（见 §6）。

2.3 能力地图

领域	已实现	状态 / 备注
解码	rsraw/LibRaw、libheif、image crate；JPEG/PNG/TIFF/WebP/BMP/GIF/HEIC/RAW	已接入；45.7MP NEF 首屏 218ms、全尺寸 4.9s［自述］
调色	15 项基础、RGB/分通道/参数曲线、HSL 8 色带、颜色分级、点颜色、相机校准、黑白混色、胶片色调映射	CPU/GPU 同源；`fdxmp-color` 37 个测试
蒙版	画笔 / 线性 / 径向 / 颜色范围 / 亮度范围；AI 主体、天空、人脸、物体；每蒙版独立全套参数	Windows/iOS 已接；移动端推理仍有缺口
修图	污点修复、仿制、液化、LaMa 局部重绘、GFPGAN 人脸增强、Real-ESRGAN 超分、姿态塑形	v1.6.0 起的差异化能力；模型按需下载
预设	`.fdxmp`（JSON + ZIP + 可选 LUT）、7 类内置、XMP 导入 25+ 字段、强度混合	四端共用；XMP 导出未完整
导出	JPEG/PNG/TIFF/WebP/HEIC；尺寸/质量；sRGB/P3/AdobeRGB/ProPhoto（lcms2）；EXIF/XMP；水印；批处理	TIFF 元数据、WebP 有损仍是已知缺口
移动端	iOS 编辑界面最完整（14.3k Swift）；Android 6.3k Kotlin；Harmony 0.7k ArkTS；iOS 未签名 IPA 云打包	Android/Harmony 编辑功能仍在追赶；iOS Metal 预览待补

2.4 工程质量：有亮点，也有“文档先行”的痕迹

从代码本身看，fdxmp 的工程质量高于“15 天项目”的平均水平：`#![deny(unsafe_code)]`、`missing_docs` 警告、错误类型统一、305 个测试、对 GPU 数学有 CPU 参考实现、对色彩修复有专门的 `COLOR_FIX_LOG.md`。`crates/color` 还是零第三方依赖、`forbid(unsafe_code)` 的纯数学 crate，这一点很适合四端复用。

但仓库的**文档与代码已经出现明显漂移**。这不是“代码错”，而是评审与交付风险：外部读者会按 README 的承诺验收，而实际版本、产物、测试状态并不总对得上。

文档声明	代码/仓库现状	影响
README 阶段表：阶段 6 “待开始”	PROJECT_REPORT 称桌面 UI、打包器、NSIS/7z 已完成	读者无法判断当前真实完成度
PROJECT_REPORT：生成于 2026-10-03、版本 0.4.0	当前 v1.6.1、最后提交 2026-10-24	测试数、功能、性能数字均已过期
PROJECT_REPORT：`dist/fdxmp-setup.exe`、APK、HAP 已产出	`dist/` 仅 `.gitkeep`	交付物不可复现/不可验证
README/架构文档：根目录 `F:\FD-Room`	实际 `D:\color-adjustment-software`	命令与路径照抄会失败
M2 记录：“全部测试通过”	同页又记录 2 个 GPU 测试基线失败、2 处 clippy 警告	质量结论自相矛盾
文档：FFI 导出 20 个 C ABI	代码中可见 40+ 个 `fdxmp_*` 导出（含条件编译）	集成方按文档接会漏接口

性能方面，文档内部也有“修正前/修正后”两套数字：README 阶段 2 的 5.4ms/帧（≈185fps）是早期数字；`COLOR_FIX_LOG` 在修正正确性后记录为 6.8–8.3ms（≈121–147fps），加两个蒙版 18–23ms（≈43–55fps）。这仍满足 ≥30fps 的目标，但说明**性能声明需要版本化，不能只留最好看的一组。**

2.5 优势与短板

维度	优势	短板
产品	唯一同时覆盖 Windows/iOS/Android/HarmonyOS 的项目；AI 修图链完整；Lightroom 式交互落地快	没有 catalog/DAM；相机色彩与 Lightroom 有硬差距；移动端编辑完成度不均
技术	纯 Rust 色彩 crate、CPU/GPU 同源、f16 线性管线、C ABI 易嵌入	工作空间是线性 sRGB，宽色域余量小于 LightCraft/darktable；rsraw 未暴露 DCP/as-shot WB
工程	305 个测试、错误模型统一、修复日志详细、`deny(unsafe_code)`	CI 只有 iOS 一条；文档漂移；核心作者集中，bus factor 低
生态	`.fdxmp` 预设、XMP 导入、MIT 许可	没有插件/Lua/MCP 级自动化；没有公开社区与发布节奏

2.6 五个关键风险

风险	概率/影响	描述	缓解
R1 相机色彩	高	C6c/C3c：rsraw 不暴露 DCP/as-shot WB，开尔文以 5500K 为相对基准；厂商色彩/肤色与 Lightroom 有差异	换可配置 LibRaw 绑定或自建 DNG 色彩引擎；以 ΔE 基线验收
R2 图库/持久化	高	只有 Engine 内存态历史 + 预设；没有 SQLite catalog、没有跨会话编辑记录/相册	引入 catalog crate；编辑参数 + 历史 + 评级 + 相册 + XMP sidecar
R3 验证与文档	高	CI 只有 iOS；docs 与代码/产物不一致；测试状态靠自述	Windows/macOS/Linux + iOS 构建矩阵；文档从代码生成；发布清单
R4 许可合规	中	LibRaw/libheif 为 LGPL/CDDL；`lensfun` crate 标注 LGPL-3.0-or-later；模型权重各有许可	逐项审计；动态链接/可重链接；NOTICE 与源码提供方式
R5 移动端性能	中	iOS Metal 预览待补；Android/Harmony GPU 路径与 AI 运行时未完全打通	平台 GPU 后端 + CPU 回退；低端设备降级；帧时间基准

3 LightCraft：25 天做出的“现代 Rust 全流程”

3.1 项目快照

项目项	现状
仓库 / 版本	`storytold/lightcraft`；v0.13.0（2026-10-23）；创建于 2026-09-30
代码	Rust 316.1k 行 / 764 个文件；29 个 crate + 3 个 app + xtask；文档约 10.8MB
测试	4,912 个 `#[test]`；强制分层、clippy 禁 unwrap/expect/panic（生产代码）、仅 sysmem 允许 unsafe
许可	MIT OR Apache-2.0（仓库同时有 LICENSE-MIT / LICENSE-APACHE）
社区	14,551 stars、5,595 forks、1,059 open issues、62 watchers（2026-10-25）
自我定位	alpha；Lightroom Classic 15.5 / Lightroom 9.6 的开源替代；原生 macOS/Windows/Linux + WebAssembly + MCP
自评完成度	功能广度 81.6%（532 项清单）；“真正可投入工作”约 61%；到 beta 约 340–600 小时

25 天、316k 行、4,912 个测试，这个速度本身就是一个需要解释的现象。仓库的贡献者列表里出现了 `claude` 账号，提交中有大量“integration batch”式合并；这至少说明它大量使用 AI 辅助开发与自动化合并。它解释了短期覆盖大量功能的速度，也带来一个现实风险：代码量的增长可能快于人类评审带宽。

3.2 架构：用“层”约束复杂度

LightCraft 的 `cargo xtask layers` 强制约束依赖方向：下层不能依赖 egui/eframe/winit/rfd，每个 crate 只能依赖更低层。对一个功能这么多的项目，这比“模块划分图”更有价值——它把架构约束变成了 CI 检查。

层	crate	职责
L0	geom / color / raster / tiff / fetch / sysmem	矩阵与 warp、色彩空间、浮点图像、TIFF、模型下载、唯一的 unsafe FFI
L1	raw / codecs / meta / develop / denoise / faces / scenes / heif	RAW 解码与去马赛克、编解码、EXIF/XMP、编辑模型、AI 降噪、人脸、演示场景
L2	pipeline	场景参考浮点管线；CPU 参考实现（oracle）
L3	catalog / gpu / preview / merge / segment	图库与持久化、wgpu 计算、缩略图缓存、HDR/全景、SAM 3 分割
L4	engine	会话、命令注册表、历史、渲染/导出任务、导入、Lightroom catalog 导入
L5	ui-egui / mcp	Lightroom 风格界面（可替换）、MCP 服务器
L6	apps / xtask / testkit	桌面、CLI、Web、工具链与测试辅助

3.3 渲染管线：五段式，宽色域，CPU/GPU 双实现

LightCraft 的 `crates/pipeline` 把管线明确分为五段：**geometry → scene-linear → tone map → colour → display**。输入是 scene-referred、线性的 Rec.2020 图像；空间参数按长边归一化，因此 400px 预览与 60MP 导出观感一致。GPU 版本是同一套阶段的 WGSL 实现，CPU 版本是 oracle，二者以 1/255 为验收口径。

阶段	内容	要点
1 几何	用户朝向、镜头校正（畸变/色差/暗角）、透视、裁剪/摆正、翻转；一次重采样；去紫边	先把几何定下来，后续空间滤镜在输出分辨率上工作
2 场景线性	白平衡、曝光、去雾、局部色调（高光/阴影）、纹理、清晰度、局部蒙版	guided filter 在 log 亮度上做边缘感知
3 色调映射	对比度/白点/黑点的 filmic 曲线、高光去饱和	只作用于亮度，尽量不偏色
4 颜色	自然饱和度、饱和度、8 色带混色、颜色分级、黑白（OkLCh）	感知空间运算，肤色保护
5 显示	色域映射、编码、参数/点曲线、暗角、颗粒	输出 sRGB 或指定空间；与显示 profile 解耦

3.4 功能广度：接近“什么都有”

领域	能力	状态
图库 / 筛选	All Photos、Recently Added、Picks、By Date、Albums/Folders、Recently Deleted、评级/旗标/色标、字段搜索、排序、虚拟化网格	●
冲图 / 对比	Compare（同步缩放）、Survey、自动前进、即时预览	●
调色	Light/Color/Effects、曲线、8 色带 Mixer、Point Color、Color Grading、Calibration、B&W	●
蒙版	画笔、线性、径向、亮度/颜色范围、Add/Subtract/Intersect；Subject/Sky 启发式；Object/Describe 需 SAM 3	◐
光学 / 几何	镜头校正、色差、去紫边、裁剪/摆正、Upright、变换	●
细节 / 修图	锐化、亮度/颜色降噪、污点修复/仿制/红眼；AI 降噪仅 Bayer 且需权重；超分缺失	◐
合成 / HDR	HDR、全景、HDR 全景 → DNG；自动对齐/去鬼影	●
输出	打印布局、接触印相 PDF、幻灯片、JPEG/PNG/TIFF/PSD/WebP/AVIF/JXL/DNG 导出	◐
自动化	命令注册表、CLI、JSON-lines 控制通道、MCP 服务器、每个控件可按名点击	●
平台	macOS/Windows/Linux/FreeBSD/Web；无移动端	◐

3.5 工程质量：这是它最值得抄的部分

维度	做法
测试密度	4,912 个测试 / 316k 行 ≈ 每 64 行一个测试；单元、属性、集成、端到端 CLI/MCP 都有
静态约束	`#![forbid(unsafe_code)]`；生产代码禁 unwrap/expect/panic/unreachable；分层检查进 CI
一致性	CPU 管线是 oracle，GPU 结果按 1/255 对比；GPU 失败自动回退 CPU

维度	做法
可观测性	CI 探测机器（OS/CPU/RAM/GPU/语料/字体），跳过项写入 `target/ci/report.json` / `summary.md`
文档纪律	`ROADMAP.md` / `docs/gaps.md` / `parity-checklist.md` 逐项列缺口与工时，甚至承认“渲染保真度未测”
自动化	所有动作是 command；UI/键盘/CLI/控制通道/MCP 同一入口；截图可由 agent 驱动

它的透明度也值得注意：`docs/gaps.md` 把“相机色彩是估计而非测量”“CR3 变体无公开样本”“测试不在 macOS/Windows 跑 CI”这类难堪的事实逐条列出，并给出工时。这种“把缺口当一等公民”的做法，比一份只报喜的阶段报告更能建立信任。

3.6 优势与短板

维度	优势	短板
产品	功能覆盖最广：图库、冲图、调色、蒙版、合成、打印、幻灯片、Web、CLI、MCP	alpha；1,059 open issues；官方自评仅约 61% 可投入工作
画质	宽色域 Rec.2020、OkLCh 调色、film/AgX 方向、CPU/GPU 1/255	相机色彩靠估计/相机 JPEG 拟合，0 个色卡实测 profile；无镜头数据库
工程	分层、测试、CI、文档、命令总线；MIT/Apache 可复用	代码量与 AI 辅助速度带来评审压力；Windows/Linux QA 薄；HDR 合并内存高
平台	macOS/Windows/Linux/FreeBSD/Web；纯 Rust 无 C（HEIF 可选）	无移动端；无插件生态；打印/地图/Book/幻灯片仍有缺口
许可	MIT OR Apache-2.0，对商业/闭源友好	依赖与资产需逐项审计；clean-room 规则限制其吸收 GPL 实现

3.7 对 fdxmp 可合法复用的资产

LightCraft 是双许可，fdxmp 已按 MIT 条款吸收了其调色算子结构（见本地 `NOTICE` 与 `crates/color/README.md`）。在此基础上，以下部分仍值得继续研究或小范围复用：

- **DNG 相机色彩模型**：ColorMatrix1/2、AsShotNeutral、ProfileHueSatMap/ProfileLookTable/ProfileToneCurve 的处理，正好补 fdxmp 的 C6c/C3c 短板。

- **阶段缓存与几何优先**：`StageCache` 的“只重算受影响阶段”与 fdxmp 的三趟管线可以互相印证。
- **catalog journal**：append-only 操作日志 + 快照 + 损坏恢复，比“直接写 JSON”更适合作为 fdxmp 图库的起点。
- **命令注册表/自动化**：如果 fdxmp 未来要做批处理、脚本或 MCP，LightCraft 的 command 抽象是现成范本。
- **测试与 CI 方法**：分层检查、GPU 跳过可见化、CPU/GPU 黄金图对比、发布前 assets 审计。

4 darktable：14 年积累的“专业暗房”

4.1 项目快照

项目项	现状
仓库 / 版本	`darktable-org/darktable`；稳定版 5.6.2（2026-10-04）；master 的 RELEASE_NOTES 已指向 5.8.0
历史 / 社区	2012-03-21 创建；13,322 stars、1,495 forks、651 open issues；每半年一个稳定版
代码	`src/` 571 个 `.c` + 341 个 `.h`，531.9k 行；97 个 iop 模块；46 个 OpenCL 内核
技术栈	C / GTK3 / SQLite / OpenCL / OpenMP / Lua；RawSpeed + LibRaw + lensfun + LittleCMS2 + Exiv2
许可	GPLv3（整体）；子模块各有许可（RawSpeed、LibRaw 等）
平台	Linux / FreeBSD / NetBSD / OpenBSD / Windows 10+ / Apple Silicon macOS 15+
自我定位	摄影工作流应用与非破坏 RAW 开发者；“**不是** Adobe Lightroom 的免费替代品”

4.2 架构：像素管线 + 模块系统

darktable 的核心是 **pixelpipe**：图像按模块顺序流过一条 32-bit 浮点管线，每个模块（iop）只负责一段处理。`src/common/iop_order.c` 定义了模块顺序与版本（legacy / v3.0 / v5.0 等），升级时通过 order 版本保持旧编辑可重放。模块不仅有 CPU 实现，通常还有 OpenCL 内核；管线支持分块（tiling）、缓存与 ROI，以控制大图内存。

层	组成	职责 / 备注
UI	GTK3 视图（lighttable / darkroom / map / print / slideshow 等）	工作流入口；也是被诟病“陈旧/复杂”的部分
图库	SQLite `library.db` + XMP sidecar + 文件系统	导入、评级、标签、集合、历史；数据库迁移不可逆
像素管线	`src/develop/pixelpipe*`、`iop_order.c`、`tiling.c`、`pixelpipe_cache.c`	模块顺序、分块、缓存、ROI
模块	`src/iop/*.c`（97 个）	每个模块：process / init / commit_params / cleanup + OpenCL 变体
GPU	OpenCL 1.2 + 46 个 `.cl` 内核	可关闭；驱动差异会导致模块回退/异常
扩展	Lua 5.4 + `lua-scripts` 子模块	导出、HDR/全景、人脸、标签、GPS 等

以 v5.0 顺序为例，管线从 `rawprepare → temperature → highlights → demosaic → denoiseprofile` 开始，经过镜头/几何/局部调整，到 `colorin` 进入场景参考，再用 `colorbalancergb`、`toneequal`、`filmicrgb/sigmoid/agx` 做颜色与色调，最后由 `colorout` 转到输出空间。这个顺序本身就是 darktable 色彩科学的核心资产：**它把“什么时候该在线性/场景参考域做什么”固化了。**

4.3 色彩科学：最深的一条护城河

模块	作用	价值
colorin	输入色彩配置/相机矩阵 → 场景参考 RGB	决定相机色彩解释；支持 DCP/ICC 输入 profile
colorbalancergb	四轮（shadows/midtones/highlights/global）+ fall-off + 白点支点 + hue/chroma 分离 + 色域压缩	比 Lightroom 的三轮分级更完整；scene-referred
toneequal	曝光域局部色调，曝光不变引导滤波	高光/阴影的“曝光感”调整，减少光晕
contrastntexture	局部对比度与纹理，边缘感知、曝光不变	5.8 新模块；替代部分 clarity 需求
filmicrgb / sigmoid / agx	显示变换：柔性高光滚降、色相保持	三条路线并存，用户按素材选择
spektrafilm	光谱胶片/相纸/扫描过程的物理仿真（5.8）	模拟颗粒、halation、扩散；差异化很强
colorout	输出色彩空间与 ICC intent	显示/打印/导出统一出口

这些模块的价值不只在“功能”，更在**可解释性与可复现性**：每个模块有参数 schema、版本、混合/蒙版、预设，编辑可以写进 XMP 并在多年后重放。LightCraft 的 `process-versions` 与 fdxmp 的 `.fdxmp` schema_version 都在解决同一个问题，但 darktable 已经经历了多轮数据库/模块迁移的实战。

4.4 功能与生态

领域	能力	状态
RAW / 相机	RawSpeed + LibRaw；相机/镜头数据库；DNG/CR3/ARW/NEF/RAF/RW2/ORF 等； tethered shooting	●
工作流	lighttable、darkroom、map、print、slideshow；集合/标签/评级/历史/样式	●
色彩	输入/输出 ICC、软打样、LUT3D、色彩查找、UCS/JzAzBz、AgX/sigmoid/filmic	●
局部	drawn/parametric/raster/AI 蒙版组合；混合模式；每模块蒙版	●
扩展	Lua 脚本、样式、预设、XMP、CLI（darktable-cli）、MCP（5.8 开发版）	●
AI	对象蒙版、降噪、超分；可选 ONNX Runtime，默认关闭，模型按需下载	◐
平台	Linux 为主，Windows/macOS 为次；无移动端；Windows 打印在 5.8 才补齐	◐

4.5 工程质量与性能

维度	做法
性能工程	OpenCL 内核 + 分块 + 像素管线缓存 + ROI + 资源限制；CPU 侧 OpenMP/SIMD；大图可跑但重模块慢
测试	CTest + `src/tests` + 独立 `darktable-tests` 子模块；更偏集成/回归，不以单元测试数量见长
发布	每半年稳定版；nightly/开发快照；AppImage/Flatpak/deb/rpm/Windows/macOS 多形态
文档	用户手册独立仓库 `dtdocs`（版本化 + Weblate 翻译）；开发者指南在 wiki；主仓文档相对少

维度	做法
治理	14 年社区、CII Best Practices、Lua 生态、翻译社区；核心维护者集中但梯队完整

4.6 优势与短板

维度	优势	短板
画质	97 个模块、scene-referred 工作流、colorbalancergb/toneequal/AgX/spektrafilm；可解释、可重放	默认配置复杂；不同模块路线重叠，新手容易迷路
RAW / 镜头	14 年样本、RawSpeed、lensfun、相机/镜头数据库；tethering	部分新格式仍缺（Apple ProRAW、Nikon HE NEF、Sony 压缩 ARW 等）
工程	OpenCL + 分块 + 缓存 + 资源限制；跨平台打包成熟	C/GTK3 维护成本高；数据库迁移不可逆；Windows/macOS 平台 bug 更多
生态	Lua、样式、XMP、CLI、MCP（5.8）、社区/文档/翻译	GPLv3 阻断闭源复用；无移动端；插件生态偏 Lua 脚本
体验	专业用户可精细控制；免费、本地、无订阅	学习曲线陡；UI 陈旧；重模块对硬件要求高

4.7 对 fdxmp：能学什么，绝不能拿什么

darktable 是 GPLv3。对 MIT 的 fdxmp 来说，**任何代码、数据文件、LUT、预设、噪声配置的复制都会导致整仓被迫 GPL。**`docs/COLOR_ENGINE_PLAN.md` 已经划了红线，这个判断是正确的。能做的只有“读公开论文/文档，自己重写数学”，并在 `NOTICE` 里记录算法来源。

- **可学**：scene-referred 的阶段划分、colorbalancergb 的参数语义（四轮/fall-off/白点支点）、toneequal 的曝光不变引导滤波思路、像素管线缓存/分块、模块参数 introspection（对自动化很有用）。
- **可学**：把“模块顺序”版本化，保证旧编辑在新版本仍能重放；这是 fdxmp 目前 `schema_version` 还没有完全解决的问题。
- **不可拿**：`data/wb_presets.json`、`noiseprofiles.json`、任何 `.c/.h/.cl`、LUT、预设、图标、文档文本。
- **不必学**：GTK3 的 UI 组织、97 个模块的堆叠方式、Lua 运行时；这些与 fdxmp 的产品定位不匹配。

5 相同点：它们其实在做同一件事

把品牌、语言、许可先放一边，三个项目在“产品哲学”和“像素管线”上高度相似。这些共同点说明：专业 RAW 编辑器的基本问题是稳定的，差别在于解决得多深、多快、多广。

5.1 产品层：非破坏、RAW、本地优先

- **非破坏编辑**：原始文件不改写；编辑是参数/历史/侧车文件（fdxmp 的 `EditParams`、LightCraft 的 `develop::Settings`、darktable 的 history stack + XMP）。
- **RAW 优先**：都把 RAW 解码、去马赛克、白平衡、相机色彩作为一等公民，而不是“把 JPEG 再压一遍”。
- **本地优先**：不依赖云、不强制账号；LightCraft 明确“no account, no cloud, no telemetry”，darktable 与 fdxmp 也是本地文件/数据库。
- **专业摄影师工作流**：导入 → 筛选 → 编辑 → 导出；都支持批量、预设、历史、评级/旗标（fdxmp 较弱）。
- **跨平台桌面**：Windows/macOS/Linux 至少覆盖两者；fdxmp 另加移动端。

5.2 技术层：场景参考 + GPU + 模块化

- **场景参考线性工作空间**：fdxmp 用 `rgba16float` 线性 sRGB；LightCraft 用线性 Rec.2020；darktable 用线性 Rec.2020。都在显示编码前保留高光余量。
- **GPU/CPU 双路径**：fdxmp 用 wgpu compute + CPU 参考；LightCraft 用 wgpu + CPU oracle（1/255）；darktable 用 OpenCL + CPU。
- **模块化管线**：fdxmp 的 geometry/base/blur/main/encode；LightCraft 的五段式；darktable 的 iop_order。三者都把“顺序”当成正确性的一部分。
- **局部蒙版**：画笔/线性/径向/范围是共同的最小集合；都支持每蒙版独立参数与叠加。
- **色彩管理**：输入 ICC/相机矩阵 → 工作空间 → 输出 ICC；都支持软打样/输出色彩空间（fdxmp 较弱）。
- **预设与互操作**：fdxmp `.fdxmp` + XMP 导入；LightCraft `.lcpreset` + XMP；darktable styles + XMP。

5.3 工程层：测试、文档、AI

- **测试/CI 作为质量门**：LightCraft 最彻底（4,912 测试 + xtask CI），darktable 用 CTest/集成测试，fdxmp 有 305 个测试但 CI 覆盖薄。

- **文档作为产品的一部分**：三者都有架构/阶段/用户文档；LightCraft 的 gaps/parity、fdxmp 的修复日志、darktable 的独立手册，都是各自文化的产物。
- **AI 正在成为标配**：fdxmp 的 AI 修图、LightCraft 的 SAM 3/人脸/降噪、darktable 5.8 的对象蒙版/降噪/超分——方向一致，成熟度不同。

5.4 “共同基因”对照表

共同基因	fdxmp	LightCraft	darktable
非破坏 + 历史	Engine + History	develop::Settings + journal	history stack + XMP/DB
RAW 解码	rsraw/LibRaw	自研纯 Rust 解码器	RawSpeed + LibRaw
工作空间	线性 sRGB f16	线性 Rec.2020	线性 Rec.2020
GPU	wgpu compute	wgpu compute + CPU oracle	OpenCL + CPU
局部蒙版	5 类 + AI	画笔/线性/径向/范围 + SAM 3	drawn/parametric/raster/AI
预设/互操作	.fdxmp + XMP 导入	.lcpreset + XMP	styles + XMP
跨平台	Win + 移动三端	macOS/Win/Linux/Web/FreeBSD	Linux/Win/macOS/FreeBSD
AI	修图链（LaMa/GFPGAN/ESRGAN）	SAM 3/人脸/降噪	对象蒙版/降噪/超分

6 差异、优缺点与综合判断

6.1 硬指标对比

维度	fdxmp	LightCraft	darktable
创建时间	2026-10-09	2026-09-30	2012-03-21
版本	1.6.1	0.13.0	5.6.2 稳定 / 5.8.0-dev
语言	Rust + Swift/Kotlin/ArkTS	纯 Rust	C / C++（少量 Objective-C++）
代码规模	55.6k 行（Rust 34.3k + 移动 21.3k）	316.1k 行 Rust	531.9k 行 C/H（不含子模块）
许可证	MIT	MIT OR Apache-2.0	GPLv3
测试	305（11 ignored）	4,912	CTest + 集成子模块

维度	fdxmp	LightCraft	darktable
平台	Windows + iOS/Android/HarmonyOS	macOS/Win/Linux/Web/FreeBSD	Linux/Win/macOS/FreeBSD
工作空间	线性 sRGB	线性 Rec.2020	线性 Rec.2020
GPU	wgpu（Metal/Vulkan/DX12）	wgpu（Metal/Vulkan/DX12）	OpenCL 1.2
图库/DAM	无（文件 + 内存历史）	有（catalog journal）	有（SQLite）
自动化	C ABI / 移动桥接	CLI / 控制通道 / MCP	Lua / CLI / MCP（5.8）
打印	仅导出	打印布局 + PDF	print 视图（Windows 5.8 补齐）
移动端	三端	无	无
社区	Gitee，小	14.6k stars，年轻	13.3k stars，14 年

6.2 能力矩阵（●强 / ◐部分 / ○弱或缺失）

能力	fdxmp	LightCraft	darktable
RAW 解码广度	◐ LibRaw 覆盖面广，但受绑定限制	◐ 自研解码器，38% 机型验证	● RawSpeed/LibRaw + 14 年样本
相机色彩 / DCP	○ 无 DCP / as-shot WB	◐ 估计/相机 JPEG 拟合，实测 profile 少	● 输入 profile + 相机矩阵 + 色卡
场景参考管线	● 线性 sRGB f16	● 线性 Rec.2020	● 线性 Rec.2020
局部蒙版	● 5 类 + 每蒙版全套参数	● 组合/嵌套 + SAM 3	● 组合/嵌套 + 参数化
AI 蒙版	◐ Windows/iOS ONNX	◐ 启发式 + 可选 SAM 3	◐ 可选 ONNX，默认关闭
AI 修图	● LaMa/GFPGAN/ESRGAN/姿态	◐ 修复/降噪，无生成式填充	◐ retouch/AI 降噪/超分
图库 / DAM	○	●	●
打印 / 输出模块	◐ 导出强，无打印	◐ 打印 PDF，无直接打印	● print 视图
自动化 / 扩展	○ C ABI 之外较少	● 命令 + CLI + MCP	● Lua + CLI + MCP
移动端	● 三端	○	○
Web	○	● WASM	○
性能	● 代理图 121–147fps［自述］	● 24MP 4ms［自述］	◐ OpenCL 好，CPU 重模块慢
成熟度	○ 15 天	◐ alpha 25 天	● 14 年
许可友好	● MIT	● MIT/Apache	○ GPLv3

6.3 雷达图：没有全能赢家

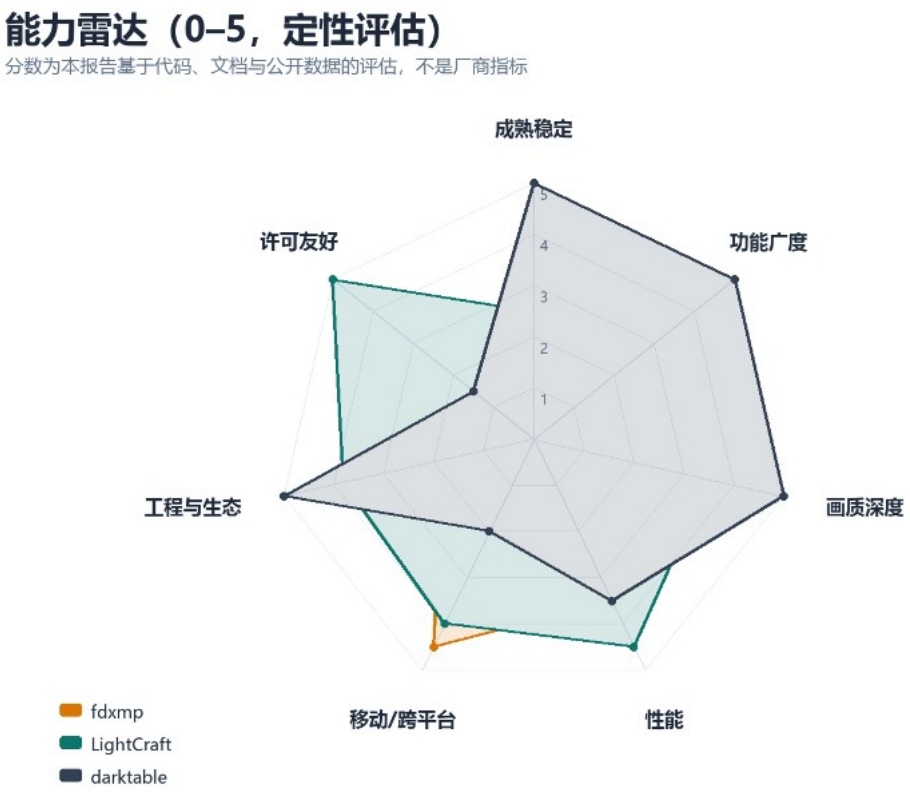


图 1 能力雷达（0–5，定性评估；分数为本报告判断）

雷达图刻意保留了三个“尖角”：darktable 在成熟度、功能、画质、工程生态上接近满分；LightCraft 在功能、性能、许可上很强，但成熟度只有 2.5；fdxmp 在移动/跨平台与许可上最高，但成熟度、画质深度、工程生态明显偏低。**平均分没有意义**，因为三者解决的场景不同。

更准确的读法是看“形状”：darktable 是右上方的大多边形，LightCraft 是横向拉长的多边形，fdxmp 是左下-右上斜向的多边形。选择哪一个，取决于你更需要哪条边，而不是谁的面积大。

6.4 fdxmp 的优点与缺点

优点

- **四端覆盖是稀缺能力**：LightCraft 和 darktable 都没有移动端；fdxmp 的 iOS/Android/HarmonyOS 工程、JNI/N-API 桥接、iOS IPA 云打包，构成了真实的产品差异。
- **AI 修图链完整**：LaMa 局部重绘、GFPGAN 人脸增强、Real-ESRGAN 超分、姿态塑形、手动修复/液化——这是“修图”而不只是“调色”的能力。

- **色彩 crate 可独立复用**：`fdxmp-color` 无第三方依赖、`forbid(unsafe_code)`、37 个测试，涵盖 CCT/OkLab/色域/单调样条/分级/校准，适合四端与 WASM。
- **Lightroom 语义落地快**：15 项、曲线、HSL、分级、点颜色、校准、黑白、蒙版、预设、历史都快齐了，交互参考 LR，学习成本低。
- **MIT + C ABI + .fdxmp**：对商业嵌入、二次开发、跨语言集成最友好；这是 GPL 的 darktable 不具备的。
- **修复日志文化**：`COLOR_FIX_LOG` 把 P0/P1、根因、回归测试写清楚，这在早期项目里很少见。

缺点

- **相机色彩是硬伤**：没有 DCP、没有 as-shot WB，开尔文是相对 5500K 的自制刻度；这意味着“同一张 RAW，fdxmp 的默认色彩与 Lightroom 不会一致”。
- **没有图库/DAM**：编辑参数与历史在内存里，缺少 SQLite catalog、跨会话编辑记录、相册/评级/批量管理；这与“对标 Lightroom”差距最大。
- **验证不足**：CI 只有 iOS 一条；305 个测试没有在 Windows/Android/Harmony 上持续跑；文档中的“全部通过”与修复日志自相矛盾。
- **移动端完成度不均**：iOS 最完整，Android/Harmony 仍在追赶；iOS Metal 预览、移动端 AI 运行时未闭环。
- **性能/内存**：全尺寸 45.7MP RAW 解码 4.9s；代理图 121–147fps 但仍受蒙版数量影响；大图导出与镜头校正仍是 CPU 路径。
- **文档与交付漂移**：版本、产物、FFI 数量、路径均有过期；`dist/` 为空让“已打包”无法复核。
- **生态薄弱**：没有插件、脚本、MCP/Lua 级自动化；社区规模小，bus factor 低。

6.5 LightCraft 的优点与缺点

优点

- **功能广度最接近 Lightroom**：图库、冲图、调色、蒙版、合成、HDR、打印、幻灯片、Web、CLI、MCP；官方清单 532 项，P0 覆盖 98%。
- **架构约束可执行**：`cargo xtask layers` 把分层变成 CI；生产代码禁 unwrap/unsafe；CPU 是 oracle，GPU 按 1/255 验收。
- **测试密度罕见**：4,912 个测试、属性测试、端到端 CLI/MCP；对 316k 行代码来说，这是它敢快速迭代的底气。
- **性能与可移植性**：纯 Rust、wgpu、WASM；24MP 滑杆约 4ms、2.5MP 冷预览约 30ms、全尺寸导出约 0.3s［自述］；无 GPU 时 CPU 回退。
- **许可友好**：MIT/Apache，可合法复用；fdxmp 已经吸收其调色算子，路径正确。

- **自动化原生**：每个菜单/滑杆/画笔都是 command，UI/CLI/控制通道/MCP 同一入口；这是“AI 时代编辑器”的范本。
- **透明度**：gaps/parity/roadmap 把缺口、工时、验证状态写清楚，比营销式 README 更可信。

缺点

- **仍是 alpha**：0.13.0、1,059 open issues、官方自评约 61% 可投入工作；稳定性、边界条件、长尾相机都需时间。
- **相机色彩/RAW 覆盖是硬缺口**：0 个色卡实测 profile；38% 机型验证；CR3 变体、压缩 ORF、Nikon HE NEF 仍可能回退到预览。
- **平台 QA 薄**：测试主要在 Linux/FreeBSD；macOS/Windows 只构建不跑测试；NVIDIA/Intel/DX12 GPU 未验证；Windows 卸载有已知 bug。
- **没有移动端/插件生态**：无法覆盖手机端；Lightroom 插件（Negative Lab Pro 等）无替代。
- **复杂度/维护风险**：316k 行、10.8MB 文档、305KB parity checklist；AI 辅助开发的速度快于人类评审，长期可维护性待观察。
- **资源占用**：HDR 合并每帧 32-bit float 约 0.86GB；release 构建在 16GB 机器上可能 OOM（issue #1196）。
- **clean-room 限制**：为了许可洁净，不能读 GPL 的 raw/photo 代码，导致某些 RAW 格式只能自己啃专利/规范，进度慢。

6.6 darktable 的优点与缺点

优点

- **画质深度**：97 个模块、scene-referred 工作流、colorbalancergb/toneequal/AgX/filmicrgb/spektrafilm；对色彩有近乎“实验室级”的控制。
- **RAW/镜头覆盖**：RawSpeed + LibRaw + lensfun + 14 年样本；tethering、DNG opcode、相机/镜头数据库，是三者中最完整的。
- **性能工程**：46 个 OpenCL 内核、分块、像素管线缓存、ROI、资源限制；大图与重模块有成熟的工程解法。
- **生态与扩展**：Lua 脚本、styles、XMP、CLI、MCP（5.8）、翻译社区、每半年发布；14 年积累的社区与文档。
- **成熟稳定**：14 年、CII Best Practices、跨平台打包（AppImage/Flatpak/deb/rpm/Windows/macOS）；旧编辑可重放。
- **免费本地**：无订阅、无云；对不愿被 Adobe 绑定的专业用户有强吸引力。

缺点

- **GPLv3**：与 MIT 的 fdxmp 不兼容；不能复制任何代码/数据，否则整仓被迫 GPL。这是对 fdxmp 最硬的一条。
- **学习曲线陡、UI 陈旧**：GTK3、模块多、参数密集；新手容易“不知道从哪开始”，与 Lightroom 的线性流程差异大。
- **资源重**：GPU 强烈推荐；denoise/local contrast/contrast equalizer/retouch/liquify 在 CPU 上可能慢到不可用；Windows/macOS 平台 bug 更多。
- **无移动端**：无法覆盖手机/平板；macOS 只支持 Apple Silicon + 较新系统。
- **C 代码维护成本**：手动内存管理、模块 API 复杂、数据库迁移不可逆；贡献门槛高。
- **AI 复杂**：可选 ONNX Runtime、模型下载、GPU provider 配置；默认关闭，普通用户难用上。
- **文档分散**：主仓文档少，用户手册在 dtdocs、Lua 文档在 luadocs、开发者指南在 wiki。

6.7 分场景结论：谁更强?

场景	首选	理由
极限画质 / 大幅打印 / 色彩研究	darktable	模块深度与 scene-referred 工作流最强；接受学习曲线。
桌面全流程 / 自动化 / 可复用代码	LightCraft	MIT/Apache + 分层 Rust + MCP/CLI；接受 alpha。
移动端 / 端侧 AI 修图 / 商业嵌入	fdxmp	四端 + AI 修图 + MIT/C ABI；先补相机色彩与图库。
教学 / 算法研究	darktable（读）+ LightCraft（写）	darktable 给问题与参数语义，LightCraft 给可运行的 Rust 实现。
闭源产品/插件集成	fdxmp 或 LightCraft	MIT/Apache 可闭源分发；darktable GPL 不适合。
长期稳定/社区支持	darktable	14 年、半年一发、社区/文档/翻译完整。

> **没有全能赢家**：darktable 赢在深度与时间，LightCraft 赢在工程与速度，fdxmp 赢在移动端与 AI。把三者当成“同一赛道的三名选手”会误判；更准确的比喻是：一个成熟的暗房、一个高速迭代的现代工作台、一个面向手机与 AI 的新物种。

7 对 fdxmp 的建议：先补地基，再谈对标

fdxmp 当前最危险的不是“功能少”，而是“功能表看起来很多，但基础不牢”。如果明天拿一台主流相机拍一张 RAW，与 Lightroom 默认渲染对比，相机色彩、镜头校正、图库工作流这三项会立刻暴露差距。因此建议按 P0/P1/P2 排优先级，而不是继续加滑杆。

7.1 P0：0–4 周，决定“能不能用”

优先级	动作	验收标准
P0-1 相机色彩	换可配置 LibRaw 绑定或接入 DNG 色彩引擎；读取 AsShotNeutral/ColorMatrix；补 DCP 或相机矩阵；把开尔文改为标准模型	3 台相机、每种 3 张 RAW，与相机 JPEG/Lightroom 默认的 ΔE2000 基线；白平衡方向正确
P0-2 图库/持久化	新增 SQLite catalog：图片、编辑参数、历史、评级、相册；XMP sidecar 读写；崩溃恢复	关闭重开恢复编辑；1 万张导入可搜索/排序；拔电不丢历史
P0-3 CI 与发布	Windows/macOS/Linux + iOS 构建矩阵；跑 305 测试 + clippy/fmt；GPU 冒烟；打包产物校验	每次 PR 全绿；`dist/` 可由 CI 重建；版本号与文档自动生成
P0-4 文档一致性	以代码/CI 为唯一事实源，重写 README 的阶段表、FFI 数量、产物清单；删除过期路径/数字	README、PROJECT_REPORT、docs 三者版本/数字一致；评审可按文档复现

7.2 P1：1–3 个月，决定“好不好用”

- **移动端 GPU 闭环**：iOS Metal（wgpu）预览、Android Vulkan 常驻会话、Harmony GPU 路径；失败自动回退 CPU，并给出可见提示。
- **1:1 与分块渲染**：全分辨率按需解码 + 可见分块；mip 金字塔解决 >12288px 的细节半径；镜头校正进 GPU 几何趟。
- **性能基准化**：把 45.7MP 首屏、24MP 滑杆、2 蒙版、全尺寸导出写成 criterion/CI 冒烟；每次发布对比基线，禁止“只报最好看的一组”。
- **镜头/相机数据库**：优先 DNG opcode 与主流镜头；lensfun 数据许可要单独核实（LGPL 数据/库的合规方式）。
- **AI 模型交付**：模型按需下载 + SHA-256 + 失败降级；移动端 Core ML/LiteRT/XEngine 至少各打通一条推理链；模型卡与许可证随包。

7.3 P2：3–6 个月，决定“能不能长大”

- **自动化/插件**：借鉴 LightCraft 的 command registry，给批处理、脚本、MCP 留统一入口；这比再做一个 UI 按钮更有杠杆。
- **打印/HDR/软打样**：如果目标是“对标 Lightroom”，打印与 HDR 显示是迟早要补的；可以后做，但架构上要预留输出色彩管理。
- **社区与发布节奏**：公开 GitHub 镜像/issue tracker、贡献指南、每 2–4 周一个可安装版本；把“修复日志”变成 release notes。
- **许可审计**：LibRaw（LGPL/CDDL）、libheif（LGPL）、lensfun（LGPL）、模型权重、字体/图标逐项列明；静态链接 LGPL 需提供可重链接方式。

7.4 不建议做的事

- **不要复制 darktable 的任何代码/数据/LUT/预设**；GPLv3 会传染，MIT 项目承受不起。
- **不要自研全部 RAW 解码器**：LightCraft 用 36.5k 行才覆盖部分机型；fdxmp 应优先把 LibRaw 的可配置能力用满，再考虑补缺。
- **不要追求“功能数量对齐 Lightroom”**：LightCraft 的 532 项清单是 25 天 + 4,912 测试 + 大量 AI 辅助才堆出来的；fdxmp 应先赢“移动端 + AI 修图”这一个场景。
- **不要在没有 CI 的情况下继续加模块**：每加一个滑杆，CPU/GPU/移动端/导出/预设/XMP 六条路径都要验证；否则技术债会指数增长。

7.5 风险登记表

风险	概率	影响	缓解
相机色彩不达标	高	高	P0-1；以 ΔE 基线而非肉眼验收
图库缺失导致用户流失	高	高	P0-2；先做最小 catalog，再做相册/评级
移动端 GPU/AI 进度	中	高	P1；每端至少一条闭环 + CPU 回退
LGPL/模型许可	中	中	P2；法务审计 + 动态链接/可重链接
文档/交付不可信	高	中	P0-4；CI 生成指标，发布清单
核心作者 bus factor	中	高	公开仓库/贡献指南/代码所有权分散

8 结论

第一，三个项目不在同一个成熟度维度上。 darktable 是 14 年积累的专业暗房，LightCraft 是 25 天冲到 0.13.0 的现代全流程 alpha，fdxmp 是 15 天、以移动端和 AI 修图为差异化的新项目。把它们的“功能数量”直接对比，会得到错误结论；更有意义的是比较“在各自阶段，是否把关键问题解决对了”。

第二，相同点多于不同点。 非破坏、RAW、场景参考线性、GPU/CPU 双路径、局部蒙版、预设/历史、跨平台、本地优先——这些是专业 RAW 编辑器的“共同基因”。真正的差异是：darktable 把色彩科学做深，LightCraft 把工程与自动化做宽，fdxmp 把移动端与 AI 修图做近。

第三，各自的优缺点都很清楚。 darktable 最强在画质深度、RAW/镜头覆盖、生态与成熟度，最弱在许可、学习曲线与资源占用；LightCraft 最强在架构、测试、功能广度、性能与许可，最弱在 alpha 稳定性与相机色彩；fdxmp 最强在四端覆盖、AI 修图、MIT/C ABI，最弱在相机色彩、图库、验证与文档一致性。

对 fdxmp 的最终判断：它现在的正确标签不是“Lightroom 替代品”，而是“有潜力的移动端 RAW 编辑器 + AI 修图工具”。路线是对的——先占移动端与 AI，再补桌面专业能力；但未来 6–12 个月必须把相机色彩、catalog、CI/文档三块地基补上。否则，功能表越漂亮，越容易被一次实拍对比打回原形。

对两个开源项目的判断：LightCraft 值得持续跟踪、提交 issue/PR，并在 MIT/Apache 许可下小范围复用；darktable 值得学习其色彩科学与工程方法，但**不能复制**。前者是“可以一起长大的邻居”，后者是“值得尊敬的老师，但教材不能抄”。

> **如果只能记三句话**：① darktable 代表深度与时间，LightCraft 代表工程与速度，fdxmp 代表移动端与 AI；② 三者共同的技术底座是“非破坏 + 场景参考 + GPU + 局部蒙版”，差别在覆盖与成熟度；③ fdxmp 的下一步不是加功能，而是补相机色彩、图库与可复现的验证。

附录 A 数据口径与明细

A.1 代码规模

项目	语言	文件数	物理行数	说明
fdxmp	Rust	58	34,279	core 42 文件/23,706 行；win_ui 6,077；ffi 2,322；color 1,639
fdxmp	Swift	26	14,312	iOS 编辑界面/桥接
fdxmp	Kotlin	18	6,272	Android Compose/桥接
fdxmp	ArkTS	4	749	HarmonyOS 页面/桥接
LightCraft	Rust	764	316,070	29 crates + 3 apps + xtask；含测试
darktable	C / C++ / headers	912	531,935	571 个 .c + 341 个 .h；不含子模块

A.2 关键版本与社区数据（2026-10-25）

项目	版本 / 发布	stars	forks	open issues	创建时间
fdxmp	1.6.1 / 本地	—	—	—	2026-10-09（首提交）
LightCraft	0.13.0 / 2026-10-23	14,551	5,595	1,059	2026-09-30
darktable	5.6.2 / 2026-10-04	13,322	1,495	651	2012-03-21

A.3 测试与质量门

项目	测试/质量门	覆盖范围	缺口
fdxmp	305 个 `#[test]`；clippy/fmt 自述通过；`deny(unsafe_code)`	core/color/FFI/GPU/解码/预设	CI 仅 iOS；11 个 ignored；无性能回归门
LightCraft	4,912 个测试；xtask CI（fmt/clippy/tests/layers/assets/WASM）	全 workspace + 端到端 CLI/MCP	macOS/Windows 不跑测试；NVIDIA/Intel/DX12 未验证
darktable	CTest + 集成测试子模块 + CI	模块/管线/像素缓存/导出	单元测试密度低；平台差异大；部分格式缺样本

附录 B 许可与复用边界

这一节不是法律意见，而是工程上必须提前想清楚的边界。三个项目的许可决定了“能不能拿代码”，而依赖/数据/模型的许可决定了“拿了之后能不能分发”。

使用方式	fdxmp（MIT）	LightCraft（MIT/Apache）	darktable（GPLv3）
复制代码进 MIT 项目	可以（保留版权/许可声明）	可以（MIT 或 Apache 条款）	**不可以**（GPLv3 传染）
链接/动态使用库	视库许可	视库许可	GPLv3 库通常要求整体 GPL
复用数据文件	需逐项核实	需逐项核实	GPL 数据不可复制
闭源分发	可以	可以	不可以（除非合规隔离/法务方案）
专利授权	MIT 无明示专利许可	Apache-2.0 有明示专利许可	GPLv3 有专利条款

fdxmp 当前的具体注意点：LibRaw 是 LGPL-2.1/CDDL，libheif 是 LGPL-3.0，`lensfun` crate 标注 LGPL-3.0-or-later；如果静态链接 LGPL 库，分发时需要提供可重链接的目标文件/源码获取方式。AI 模型权重（LaMa Apache-2.0、GFPGAN MIT、Real-ESRGAN BSD-3 等）也要在 NOTICE/模型卡中逐项列明。

LightCraft 的许可策略值得学习：它把“代码许可（MIT/Apache）”“品牌（trademark）”“资产（ATTRIBUTION）”“字体（独立仓库）”分开管理；这比在 README 里写一句“开源”要专业得多。fdxmp 已有 `NOTICE`，下一步是把依赖/模型/字体也纳入同一张表。

附录 C 关键文件索引

C.1 本地 fdxmp

主题	路径
架构	docs/ARCHITECTURE.md、docs/SPEC.md
色彩方案	docs/COLOR_ENGINE_PLAN.md、docs/COLOR_FIX_LOG.md、crates/color/README.md
核心管线	crates/core/src/pipeline.rs、color_engine.rs、tone.rs、gpu_pipeline.rs
着色器	crates/core/src/shaders/color_pipeline.wgsl、blur.wgsl、geometry.wgsl
蒙版/AI	crates/core/src/masks.rs、smartmask/、retouch/、docs/AI_MASKS.md、docs/AI_RETOUCH.md
预设/导出	crates/core/src/preset.rs、export.rs、params.rs
FFI	crates/ffi/src/lib.rs、docs/FFI.md
阶段报告	docs/STAGE0_REPORT.md … STAGE5_REPORT.md、PROJECT_REPORT.md

C.2 LightCraft

主题	路径
架构	docs/architecture.md、Cargo.toml、crates/*/Cargo.toml
管线	crates/pipeline/src/lib.rs、colorops.rs、tone.rs、finish.rs
GPU	crates/gpu/src/render.rs、docs/gpu-pipeline.md
图库	crates/catalog/src/journal.rs、docs/lightroom-catalog-import.md
完成度	ROADMAP.md、docs/gaps.md、docs/parity-checklist.md
自动化	docs/control-protocol.md、docs/mcp.md、crates/mcp/
RAW	crates/raw/src/vendor/*、docs/raw-parity.md

C.3 darktable

主题	路径
管线	`src/develop/pixelpipe.c`、`pixelpipe_hb.c`、`tiling.c`、`pixelpipe_cache.c`
模块顺序	`src/common/iop_order.c`（legacy / v3.0 / v5.0）
模块	`src/iop/*.c`（97 个）
色彩科学	`src/iop/colorin.c`、`colorbalancergb.c`、`toneequal.c`、`filmicrgb.c`、`agx.c`、`sigmoid.c`、`colorout.c`
OpenCL	`data/kernels/*.cl`（46 个）
发布	`RELEASE_NOTES.md`、`README.md`、`.gitmodules`
测试	`src/tests/`、`src/tests/integration`（子模块）

附录 D 参考资料

- **LightCraft 仓库：** https://github.com/storytold/lightcraft
- **LightCraft 架构：** https://github.com/storytold/lightcraft/blob/main/docs/architecture.md
- **LightCraft 路线图：** https://github.com/storytold/lightcraft/blob/main/ROADMAP.md
- **LightCraft 缺口清单：** https://github.com/storytold/lightcraft/blob/main/docs/gaps.md
- **LightCraft v0.13.0：** https://github.com/storytold/lightcraft/releases/tag/v0.13.0
- **darktable 仓库：** https://github.com/darktable-org/darktable
- **darktable 发布说明：** https://github.com/darktable-org/darktable/blob/master/RELEASE_NOTES.md
- **darktable 5.6.2：** https://github.com/darktable-org/darktable/releases/tag/release-5.6.2
- **darktable 用户手册：** https://docs.darktable.org/usermanual/5.6/en/
- **fdxmp 本地文档：** `docs/`、`PROJECT_REPORT.md`、`crates/color/README.md`

说明：GitHub 数据来自 REST API 与仓库快照；本地 fdxmp 数据来自 git 历史与源码统计。报告中的引用时间为 2026-10-25（Asia/Shanghai）。